

---

# Episodic Memory-Guided Controllable Experience Synthesis for Reinforcement Learning

---

Xiao Ma<sup>1 2</sup> Tian Li<sup>2 3</sup> Wu-Jun Li<sup>2 3</sup>

## Abstract

In real-world scenarios, data collection for reinforcement learning (RL) is often constrained by safety concerns and high costs, resulting in limited data availability. Diffusion models (DMs) have recently demonstrated remarkable capabilities in capturing complex distributions, making data augmentation a promising approach. However, existing DM-based data augmentation methods still suffer from the limited quality of synthesized data for downstream RL tasks. To overcome this limitation, we propose a novel method called episodic memory-guided controllable experience synthesizer (EMCES). EMCES incorporates an episodic memory-based controllable DM with informative yet concise conditions constructed by episodic memory (EM). To guide the synthesis toward high-quality data, we propose an EM-prioritized condition sampling strategy that leverages EM-based temporal-difference errors to focus generation on data most helpful for RL. Furthermore, we introduce a hashing-based state representation for EM to improve its efficiency and further boost the quality of synthetic data. To the best of our knowledge, EMCES is the first work to incorporate EM into controllable DMs and to leverage EM for guiding data synthesis in RL. Experimental results across multiple environments demonstrate that EMCES significantly improves the quality of the synthetic data, thereby improving the performance of several state-of-the-art RL algorithms. In particular, the hashing-based state representation can reduce storage cost by about 8000-fold and reduce time cost by 25.5-fold, without degrading the normalized score.

## 1. Introduction

Reinforcement learning (RL) (Sutton & Barto, 2018) has achieved significant progress across a variety of domains, including game playing (Silver et al., 2016), robotics (Miki et al., 2022), and large language models (Ouyang et al., 2022). Due to its trial-and-error learning paradigm, RL typically requires large amounts of data obtained through extensive interactions with the environment to learn effective policies. However, collecting sufficient data in real-world scenarios is frequently constrained by safety concerns and high costs, resulting in a persistent data scarcity problem in RL (Levine et al., 2020). To alleviate this problem, researchers have focused on improving the sample efficiency of RL algorithms (Lin et al., 2018; Mnih et al., 2015; Schaul et al., 2016; Horgan et al., 2018). This problem has also driven the development of offline RL (Levine et al., 2020), where an agent learns a policy from a fixed dataset without direct interactions with the environment. Despite these advancements, data scarcity remains a fundamental challenge in RL. Inspired by the success of data augmentation in computer vision (He et al., 2023b; Trabucco et al., 2024), various data augmentation methods have been introduced into RL, including random augmentation (Laskin et al., 2020; Sinha et al., 2021), generative model-based augmentation methods (Hu et al., 2024; Lu et al., 2023b; Yang & Wang, 2025) and so on. Compared to random augmentation, data synthesized by generative models, particularly diffusion models (DMs), have exhibited higher fidelity (Lu et al., 2023b; Yang & Wang, 2025), meaning they more closely match real-world scenarios. As a result, they have greater potential for improving the performance of RL algorithms.

DM-based augmentation methods can be categorized into two types: trajectory-level and transition-level. Compared with trajectory-level methods (Yang & Wang, 2025; Li et al., 2024; Yang & Wang, 2024; Lee et al., 2024), transition-level methods (Lu et al., 2023b; Wang et al., 2025) are easier to generate high-fidelity data for RL. High-fidelity data implies that the generated transitions faithfully capture the environment dynamics, which is the foundation for effective policy learning. Hence, we focus on the transition-level DM-based augmentation methods. However, the quality of the generated data for policy learning still remains limited.

---

<sup>1</sup>School of Computer Science and Technology (School of Artificial Intelligence), Zhejiang Sci-Tech University, P.R.China  
<sup>2</sup>State Key Lab. for Novel Software Technology, Nanjing University, P.R.China <sup>3</sup>School of Computer Science, Nanjing University, China. Correspondence to: Wu-Jun Li <liwujun@nju.edu.cn>.

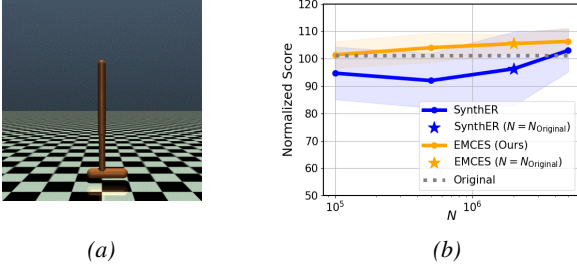


Figure 1. (a) A snapshot of Hopper. (b) Results of TD3+BC using the synthetic dataset of varying sizes on Hopper with the ‘medium-expert’ dataset. Please note that the blue and orange stars indicate the normalized scores based on synthetic data generated by Synther and EMCES, respectively, with the dataset size aligned with that of the original dataset. The grey dotted line indicates the normalized score using the original dataset.

To demonstrate this limitation more clearly, we evaluate the performance of an agent trained on synthetic datasets using TD3+BC (Fujimoto & Gu, 2021), a classic offline RL algorithm. We conduct experiments in the Hopper environment with the ‘medium-expert’ dataset, as shown in Fig. 1a. The original ‘medium-expert’ dataset from the D4RL benchmark (Fu et al., 2020) contains approximately 2M transitions pre-collected directly from the environment. The synthetic datasets are generated by Synther (Lu et al., 2023b), a classical transition-level DM-based augmentation method for offline and online RL, and range in size from 100K to 5M transitions. Figure 1b presents the normalized scores of policies trained only on synthetic datasets or the original dataset. All results are averaged over five random seeds. We observe that although the synthetic dataset matches the original dataset in size, the performance of the learned policy remains inferior. This indicates that the synthetic dataset lacks sufficient informative transitions to match the quality of the original dataset. Since Synther (Lu et al., 2023b) does not offer mechanisms to control the quality of the synthetic data, the synthetic dataset must be substantially larger than the original dataset to ensure adequate coverage of high-quality transitions. These observations highlight a fundamental limitation of current transition-level DM-based augmentation methods: the lack of controllability in the synthesis process, which hinders prioritizing high-quality transitions. They also illustrate the necessity of introducing controllable diffusion models (CDMs) (Ho & Salimans, 2021) for transition-level DM-based augmentation methods.

Episodic memory (EM) plays a crucial role in storing and integrating valuable past experiences (Andersen, 2007; Blundell et al., 2016; Lin et al., 2018; Pritzel et al., 2017), providing direct access to high-quality past experiences and thereby improving the sample efficiency of RL algorithms. These valuable experiences stored in EM would provide a possibility for a controllable synthesis process and guide the generation of high-quality data. In this paper, we propose a

novel method called episodic memory-guided controllable experience synthesizer (EMCES) to improve the quality of synthetic data. The main contributions of this work are outlined as follows:

- In EMCES, we propose an EM-based CDM, which leverages EM to construct informative but concise conditions.
- During the data synthesis process, we propose an EM-prioritized condition sampling strategy, which utilizes our proposed EM-based temporal-difference errors to prioritize generating high-quality data by guiding the EM-based CDM.
- We propose a hashing-based state representation for EM, which can improve the efficiency of EM and further boost the quality of synthetic data.
- To the best of our knowledge, EMCES is the first work to embed EM into CDMs and to leverage EM for guiding data synthesis in RL.
- Experimental results across multiple environments demonstrate that EMCES can effectively improve the quality of the synthetic data. In particular, the hashing-based state representation can reduce storage cost by about 8000-fold and reduce time cost by 25.5-fold, without compromising the normalized score.

## 2. Related Work

### 2.1. Diffusion Models in RL

DMs have advanced in RL due to their powerful and flexible distributional modeling ability (Zhu et al., 2023). DMs can be applied across various roles, including policies, planners, and data synthesizers. When used as policies, DMs generate actions conditioned on states (Wang et al., 2023; Chen et al., 2023; Lu et al., 2023c), helping to alleviate issues such as over-conservatism and limited expressiveness in offline RL. When used as planners, DMs combine policy and dynamics modeling to produce multi-step plans or full trajectories (Janner et al., 2022; Zhu et al., 2024; He et al., 2023a; Dong et al., 2024). When used as data synthesizers, DMs can be employed as data augmentation to alleviate data scarcity, as discussed in the following section.

### 2.2. Data Augmentation in RL

Data augmentation in RL can be viewed as a way to alleviate data scarcity. Traditional data augmentation methods expose the agent to multiple views of the same observation to improve the robustness of RL algorithms by adding noise injection or random transformations to raw observations (Laskin et al., 2020; Sinha et al., 2021). In contrast, researchers try

to utilize model-based imaginary samples to augment the offline dataset (Wang et al., 2021; Zhang et al., 2024). Another line of research focuses on synthesizing data using generative models (Lian et al., 2023; Huang et al., 2017), such as variational autoencoders (VAEs) (Kingma & Welling, 2014), generative adversarial networks (GANs) (Goodfellow et al., 2014) and DMs (Ho et al., 2020). In particular, due to the powerful distributional modeling capabilities of DMs (Ho et al., 2020; Karras et al., 2022; Ho & Salimans, 2021), DM-based augmentation methods have recently succeeded in synthesizing high-fidelity data for RL. DM-based augmentation methods can be categorized into two types: trajectory-level and transition-level. Compared with trajectory-level methods (Yang & Wang, 2025; Li et al., 2024; Yang & Wang, 2024; Lee et al., 2024), transition-level methods (Lu et al., 2023b; Huang et al., 2025; Wang et al., 2025) are easier and more flexible to generate high-fidelity data for RL. SynthER (Lu et al., 2023b) is a classical transition-level DM-based augmentation method that can be easily employed in various RL settings, including both offline and online RL. CFDG (Huang et al., 2025) focuses on offline-to-online RL and significantly enhances the generation quality of offline and online data with different distributions. PGR (Wang et al., 2025) only focuses on online RL and uses curiosity or value to guide the data synthesis to improve sample efficiency given an explicit value function. However, existing transition-level data augmentation methods still produce synthesized data of insufficient quality, in the sense that the generated transitions are not sufficiently informative for policy learning, especially in settings without an explicit value function.

### 2.3. Episodic Memory

Humans can rapidly exploit newly discovered high-reward opportunities due to the hippocampus, which stores EM (Squire, 2004; Andersen, 2007). Inspired by this, recent works integrate the EM mechanism into RL to help agents utilize valuable past experiences stored in EM via a non-parametric approach, thereby improving the sample efficiency (Blundell et al., 2016; Pritzel et al., 2017; Lin et al., 2018; Kuznetsov & Filchenkov, 2021; Zheng et al., 2021; Ma & Li, 2023; 2024). Specifically, the EM mechanism maintains a lookup table  $Q^S$ , where each entry  $Q^S(h(s))$  stores a tuple  $(h(s), e(h(s)))$ , with  $h(s)$  serving as the index for lookup.  $h(s)$  represents the encoded state of  $s$ , where  $h(\cdot) : \mathcal{S} \rightarrow \mathbb{R}^K$  is a state representation function, and  $K$  denotes the dimension of the encoded state.  $e(h(s))$  represents the historical optimal discounted return associated with  $h(s)$ . Given any trajectory  $\tau = \{(s_t, a_t, r_t, s'_t)\}_{t=0}^T$ , we compute the discounted return  $d(s_t) = \sum_{i=t}^T \gamma^{i-t} r_i$  at time-step  $t \in \{0, \dots, T\}$ , where  $s_t, a_t, r_t$ , and  $s'_t$  are the corresponding state, action, reward, and next state at time-step  $t \in \{0, \dots, T\}$ , respectively.  $d(s_t)$  can be abbreviated

as  $d_t$ . Then, the lookup table  $Q^S$  is updated with the set  $\{(h(s_t), d_t)\}_{t=0}^T$  based on the following rules:

$$Q^S(h(s_t)) \leftarrow \begin{cases} (h(s_t), d_t), & \text{if } Q^S(h(s_t)) \notin Q^S; \\ (h(s_t), e(h(s_t))), & \text{if } Q^S(h(s_t)) \in Q^S \text{ and } e(h(s_t)) \geq d_t; \\ (h(s_t), d_t), & \text{if } Q^S(h(s_t)) \in Q^S \text{ and } e(h(s_t)) < d_t. \end{cases} \quad (1)$$

The specific construction of EM mechanisms follows prior work (Ma & Li, 2023; Kuznetsov & Filchenkov, 2021), and readers can find it in the Appendix. Furthermore, researchers have proposed a class of parametric EM mechanisms (Hu et al., 2021; Ma et al., 2022). However, these parametric EM mechanisms are inherently limited by their reliance on Q-value functions. To the best of our knowledge, SfBC (Chen et al., 2023) is the only work that integrates both the diffusion model and EM mechanism, but it does not utilize the EM mechanism to control CDMs. Hence, our work is the first to introduce the EM mechanism as a control mechanism for CDMs.

#### 2.3.1. STATE REPRESENTATION FOR EM

Although state representation has been studied in RL (Schrittwieser et al., 2020; Hafner et al., 2020; Tang et al., 2017), state representation for EM mechanisms is still naive. The default state representation is random projection (RP), a classical dimensionality reduction technique (Kononenko & Kukar, 2007). Each state is encoded as a  $K$ -dimensional vector of real values. However, real-valued encoded states are unlikely to repeat (Li et al., 2023), leading to inefficient EM updates and high storage, retrieval, and construction costs. Grid-based state representation for EM mechanisms (Li et al., 2023) is proposed to address these issues, where each dimension of states is encoded by discretizing the state space into a grid-like structure. Grid-based state representation groups states that share the same characteristics into a single cluster, enabling more effective EM updates while reducing the complexities of storage, retrieval, and construction. However, the above state representations are both data-independent and ignore the underlying data distribution, leading to inefficient updates in the EM mechanism. These limitations motivate us to design an effective state representation for EM mechanisms.

## 3. Methodology

In this section, we introduce EMCES, a novel method to improve the quality of synthetic data. EMCES consists of three key components: (1) an EM-based CDM constructed by informative yet concise conditions, (2) an EM-prioritized condition sampling strategy to guide the EM-based CDM toward generating high-quality data, and (3) a hashing-based state representation that enhances the efficiency of the EM

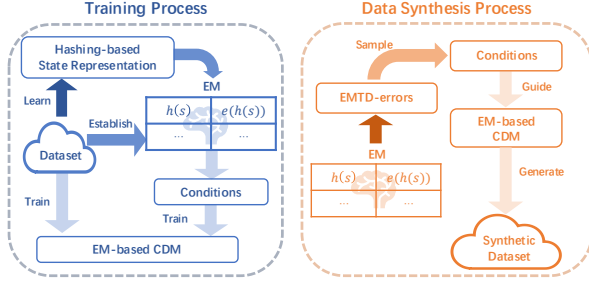


Figure 2. Architecture of EMCES during the training and data synthesis processes. Best viewed in color. For each process, darker arrows represent earlier execution steps, while lighter arrows represent later execution steps.

mechanism and further improves synthesis quality. Figure 2 shows the framework of EMCES.

### 3.1. EM-based CDM

A controllable synthesis process is crucial for enhancing the quality of synthetic data, as it enables targeted data generation and improves diversity and relevance for downstream RL tasks. Hence, we introduce CDM (Ho & Salimans, 2021) into our work, and the desired output of CDM is the transition  $x(s, a) = (s, a, r, s')$ , which represents the fundamental unit of data in RL. To capture the data distribution of the given dataset  $D$ , we train a CDM to solve the following problem:

$$\min_{\phi} \mathbb{E}_{x \in D, \sigma, \epsilon \sim \mathcal{N}(0, \sigma^2 I)} \|M_{\phi}(x + \epsilon, y(x); \sigma) - x\|_2^2, \quad (2)$$

where  $M_{\phi}(x, y; \sigma)$  is a denoiser with parameters  $\phi$ ,  $\sigma$  is the noise, and  $y(x)$  is the corresponding condition.

The effectiveness of controllable generation heavily relies on the quality of the condition design (Zhang et al., 2023). A well-specified condition  $y(x)$  enables the model to produce outputs that align with the intended semantics or dynamics. Hence, the conditions must be carefully designed to encode sufficient information about the target transition, ensuring the generation of desired outputs. To improve training efficiency, the condition should be compact yet informative. Since the state  $s$  determines the possible actions, rewards, and subsequent transitions, it must be incorporated into the condition  $y(x)$ . However, directly employing the raw state representation  $s$  might reduce the training efficiency of CDM, as it often contains redundant information, especially in high-dimensional visual states. To address this issue, we adopt a simple yet compact state representation function  $h(\cdot)$  to encode the states, and incorporate the encoded representations into the conditioning input. The specific state representation will be described in the following section. While maintaining compactness, the condition should also capture richer contextual information, such as the corresponding action, reward, next state, and potential future

return. To balance informativeness and compactness, we consider using the state-action value function  $Q(s, a)$  (or  $Q(h(s), a)$ ) to construct the term  $r + \gamma \arg \max_a Q(s', \cdot)$  (or  $r + \gamma \arg \max_a Q(h(s'), \cdot)$ ), which incorporates the corresponding action, reward, next state, and potential future return. Conventional estimation of the state-action value function  $Q(s, a)$  relies on neural networks, which demand additional pre-training and often suffer from instability (van Hasselt et al., 2016; Fujimoto et al., 2018; Lin et al., 2018; Ren et al., 2021). Hence, we introduce an EM mechanism to estimate the state-action value function  $Q(s, a)$  (or  $Q(h(s), a)$ ), leveraging its non-parametric nature to achieve stable value estimation without the need for extra training (Lin et al., 2018; Zheng et al., 2021; Ma & Li, 2023). To ensure consistency and efficiency, the EM mechanism uses the same state representation function  $h(\cdot)$  as that used in the CDM conditions.

In summary, the condition  $y(x(s, a), h(\cdot))$  for the transition  $x(s, a)$  is defined as follows:

$$y(x(s, a), h(\cdot)) := [h(s), r + \gamma e(h(s'))], \quad (3)$$

where  $h(s)$  is the compact state encoding.  $r + \gamma e(h(s'))$  denotes the EM-estimated optimal discounted return, which implicitly captures the key elements of a transition and the highest future return potentially by retrieving  $e(h(s'))$  from the lookup table  $Q^S$  in EM. Hence, this CDM is referred to as the EM-based CDM.

#### 3.1.1. TRAINING OF EM-BASED CDM

To obtain the EM-based CDM, we first construct its training dataset. Given a dataset  $D$  and a state representation function  $h(\cdot)$ , we establish an EM mechanism<sup>1</sup>. Based on this, we build a condition set  $C = \{y(x(s, a), h(\cdot)) | x(s, a) \in D\}$ . The dataset  $D$ , paired with its corresponding condition set  $C$ , is then used to optimize the loss function defined in Equation (2). The left part of Figure 2 illustrates the training procedure of the EM-based CDM. After training, the EM-based CDM can serve as an experience synthesizer, generating data conditioned on  $y(x(s, a), h(\cdot))$ . For sampling, we employ the EDM method (Karras et al., 2022).

### 3.2. EM-prioritized Condition Sampling

Although the EM-based CDM can be used directly as an experience synthesizer, its primary strength lies in generating high-quality data for RL algorithms. Intuitively, the data synthesis process should not only adhere to the underlying data distribution but also prioritize the generation of data that maximally benefits the agent’s training. While generating high-return transitions may intuitively boost performance (Liu et al., 2024), this approach often yields brittle

<sup>1</sup>Please note that the establishment of the EM mechanism can be found in the Appendix.

policies (Schaul et al., 2016), especially when trained on sub-optimal datasets. Prioritizing informative transitions has been shown to enhance both sample efficiency and stability in off-policy learning (Schaul et al., 2016; Fan et al., 2025; Wang et al., 2017). In particular, transitions with large temporal-difference (TD) errors are recognized as highly valuable, as they highlight regions where the Q-function has space for improvement (Schaul et al., 2016).

Motivated by this insight, we propose an EM-based TD error (EMTD-error) and an EM-prioritized condition sampling strategy. EMTD-error, denoted as  $\delta(x(s, a), h(\cdot))$ , serves as a principled measure of the importance of the transition  $x(s, a)$  for policy improvement and is defined as follows:

$$\delta(x(s, a), h(\cdot)) = r + \gamma e(h(s')) - e(h(s)), \quad (4)$$

where  $e(h(s))$  and  $e(h(s'))$  are retrieved from  $\mathbf{Q}^S$  by indexing  $h(s)$  and  $h(s')$ , respectively. The magnitude of the EMTD-error indicates how far the value based on the next state  $r + \gamma e(h(s'))$  deviates from the historical optimal discounted return  $e(h(s))$  of the current state  $s$ . Hence, the EMTD-error serves as an indicator of the potential utility of the transition for policy improvement. A higher EMTD-error indicates that the transition could yield greater returns than previously observed ones from the same state representation, making it more informative for policy improvement. To avoid over-sampling a small subset of transitions with high EMTD-errors, we propose an EM-prioritized condition sampling strategy to balance informativeness with diversity. Specifically, we apply a softmax operator over the EMTD-errors  $\delta(x(s, a), h(\cdot))$  to compute a sampling probability for each condition  $y(x(s, a), h(\cdot))$ , shown as follows:

$$p(y(x(s, a), h(\cdot))) = \frac{\exp^{\beta \delta(x(s, a), h(\cdot))}}{\sum_{x(s, a) \in D} \exp^{\beta \delta(x(s, a), h(\cdot))}}, \quad (5)$$

where  $\beta \in [0, 1]$  determines the level of prioritization. A higher value of  $\beta$  increases the emphasis on transitions with larger EMTD-errors, while  $\beta = 0$  corresponds to uniform sampling. Hence, based on this EM-prioritized condition sampling strategy, we guide the EM-based CDM to generate high-quality data while preserving diversity, ensuring robust policy learning.

### 3.3. Hashing-based State Representation

A well-designed state representation facilitates the construction of an effective EM mechanism by aggregating states across different trajectories and reducing both storage and time costs. In the context of EM-based CDM, it also facilitates the construction of compact yet informative conditions, which are essential for effective training and guiding the synthesis of high-quality data. However, existing state representation methods for EM are data-independent, which

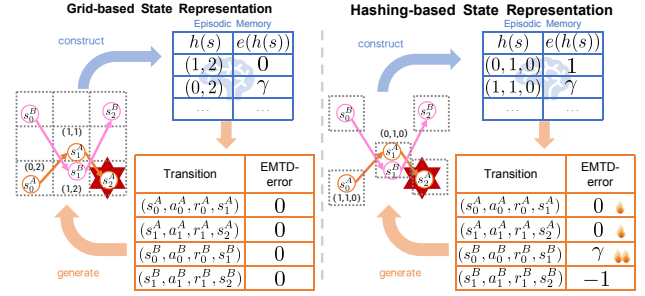


Figure 3. Illustrative example of different state representations with two trajectories  $\tau^A = (s_0^A, a_0^A, r_0^A, s_1^A, a_1^A, r_1^A, s_2^A)$  and  $\tau^B = (s_0^B, a_0^B, r_0^B, s_1^B, a_1^B, r_1^B, s_2^B)$ . Each circle and each grey box represents a state and a code, respectively. Agent in the area of the red star receives a reward of 1; otherwise, 0.

limits the efficiency of the EM mechanism. These limitations motivated us to design a novel state representation for the EM mechanism.

To improve the efficiency of the EM mechanism, we introduce a hashing-based state representation that leverages a learning-to-hash approach to encode states with dimension of  $F$  into compact and informative binary codes. Specifically, it learns  $K \leq F$  real-valued projection functions  $f(s) = \{f_k(s)\}_{k=1}^K$  given the dataset  $D$ , where each function outputs a real value and  $K$  denotes the length of the hash codes. Each projected dimension is then quantized into one bit using a hash function  $h_k(s) = \text{sgn}(f_k(s))$ ,  $k \in \{1, \dots, K\}$ , where  $\text{sgn}(x) = 1$  if  $x \geq 0$ , and 0 otherwise. We adopt IsoHash (Kong & Li, 2012), a state-of-the-art unsupervised learning-to-hash approach, to learn these projection functions. For details about IsoHash, please refer to the original paper (Kong & Li, 2012). Because hash codes are learned from the data distribution, they can align with the environment’s underlying structure, reducing aliasing between unrelated states. Furthermore, the hashing-based state representation attributes similar states with the same hash code, as shown in Figure 3, which is helpful in constructing a more effective EM mechanism by implicitly merging multiple trajectories. This state representation also provides compact and sufficient conditions for an effective EM-based CDM. Please note that the information retrieved from the EM mechanism is not used directly; instead, it is combined with the current transition’s state and action (e.g., Equation (3) and Equation (4)). This design ensures that our method remains effective even in state-sensitive tasks.

#### 3.3.1. COMPLEXITY ANALYSIS

For the EM mechanisms, its corresponding storage<sup>2</sup>, retrieval time, and construction time complexities are

<sup>2</sup>In this paper, we only focus on the storage complexity of encoded states in EM mechanisms.

Table 1. Overall results on D4RL environments using selected state-of-the-art offline RL algorithms. Note that for each task, all methods are evaluated on datasets of equal size.

| Environ-<br>ment | Task   | TD3+BC    |            |              | IQL       |           |              | EDAC         |            |              |
|------------------|--------|-----------|------------|--------------|-----------|-----------|--------------|--------------|------------|--------------|
|                  |        | Original  | SynthER    | EMCES        | Original  | SynthER   | EMCES        | Original     | SynthER    | EMCES        |
| Half<br>Cheetah  | random | 11.4±1.2  | 11.5±0.6   | 12.5±0.4     | 9.3±4.2   | 13.8±1.2  | 13.7±1.8     | 17.2±10.5    | 16.5±5.9   | 18.6±1.2     |
|                  | medrep | 44.8±0.6  | 44.5±0.7   | 45.2±0.6     | 44.5±0.2  | 44.3±0.3  | 44.3±0.2     | 64.6±2.1     | 55.8±2.4   | 56.5±0.5     |
|                  | medium | 48.1±0.2  | 48.7±0.3   | 48.8±0.2     | 48.3±0.2  | 48.9±0.3  | 49.3±0.1     | 67.5±1.0     | 63.9±2.5   | 68.2±1.0     |
|                  | medexp | 90.8±6.0  | 76.2±13.4  | 90.7±3.3     | 94.8±0.5  | 92.1±5.2  | 93.8±1.2     | 79.7±9.5     | 63.7±7.6   | 75.8±10.0    |
| Walker2d         | random | 0.4±0.3   | 1.8±2.4    | 4.0±2.5      | 3.2±1.1   | 3.4±1.1   | 3.7±0.3      | 7.0±6.9      | 10.8±8.8   | 21.6±1.1     |
|                  | medrep | 85.6±4.0  | 85.9±1.8   | 85.7±2.3     | 82.2±3.0  | 82.0±8.8  | 82.4±2.1     | 83.1±1.3     | 74.9±1.4   | 72.6±0.3     |
|                  | medium | 82.7±4.8  | 84.1±0.3   | 84.2±1.1     | 80.9±3.2  | 83.0±4.0  | 86.1±2.1     | 90.4±1.6     | 88.2±1.0   | 89.4±1.0     |
|                  | medexp | 110.0±0.4 | 110.5±0.5  | 110.7±0.3    | 111.7±0.9 | 111.2±1.0 | 111.7±0.5    | 113.5±0.5    | 111.6±1.0  | 112.9±0.6    |
| Hopper           | random | 8.7±0.5   | 7.8±0.6    | 14.1±8.7     | 7.3±0.3   | 7.9±1.0   | 7.4±0.2      | 6.2±1.6      | 5.2±1.1    | 16.4±12.2    |
|                  | medrep | 64.4±21.5 | 58.9±14.0  | 68.5±14.1    | 97.4±6.4  | 42.3±37.9 | 101.0±0.9    | 100.1±0.9    | 101.0±0.7  | 99.6±0.8     |
|                  | medium | 60.4±3.5  | 57.3±2.6   | 61.5±2.8     | 67.6±3.8  | 65.9±2.3  | 67.4±2.7     | 101.6±1.2    | 100.0±4.3  | 100.4±1.4    |
|                  | medexp | 101.2±9.1 | 96.3±13.5  | 105.5±3.3    | 107.4±7.8 | 85.2±27.0 | 102.9±8.8    | 106.7±4.0    | 99.8±14.8  | 105.0±7.0    |
| sum              |        | 708.5     | 683.5      | <b>731.4</b> | 754.6     | 680.0     | <b>763.7</b> | <b>837.7</b> | 791.4      | <b>837.0</b> |
| maze2d           | umaze  | 29.4±12.3 | 37.0±26.8  | 41.9±3.1     | 42.1±0.6  | 40.4±2.4  | 40.4±1.2     | 104.8±21.9   | 92.1±11.0  | 92.2±3.6     |
|                  | medium | 59.5±36.3 | 50.8±32.8  | 87.7±47.3    | 34.9±2.7  | 35.0±1.7  | 35.1±0.7     | 62.7±8.5     | 61.7±16.1  | 95.6±27.9    |
|                  | large  | 97.1±25.4 | 105.5±52.3 | 102.3±44.7   | 61.7±3.5  | 59.4±6.3  | 66.1±2.2     | 134.2±32.1   | 123.2±25.2 | 144.7±20.2   |
| sum              |        | 186.0     | 193.3      | <b>231.9</b> | 138.7     | 134.8     | <b>141.6</b> | 301.7        | 277.0      | <b>332.5</b> |

$\mathcal{O}(cKn)$ ,  $\mathcal{O}(\log n)$  and  $\mathcal{O}(n \log n)$ , respectively. Here,  $n$  denotes the number of distinguished encoded states given a dataset  $D$  of size  $N$ , and  $c > 0$  is the number of bits required per dimension, and  $K$  denotes the dimension of encoded states. The subscripts of  $n$  and  $c$  refer to the specific state representation. Following prior works (Blundell et al., 2016; Lin et al., 2018; Ma & Li, 2023), the lookup tables in EM mechanisms are all implemented using the KD-tree data structure (Bentley, 1975). With a fixed  $K$ ,  $n_{RP}$  is typically on the same order as  $N$ , while  $n_{Grid} \ll N$  and  $n_{Hash} \ll N$ . Hence, the hashing-based state representation can reduce the retrieval time and construction time complexities compared with the RP-based state representation. Moreover, because  $c_{Hash} = 1$  in our hashing-based state representation, whereas  $c_{RP} = 32^3$  and  $c_{Grid} \geq 1$ , the hashing-based state representation yields a notable reduction in storage complexity.

### 3.4. Adaptive to Different RL Settings

During the training process, given a dataset  $D$ , we first learn a hashing-based state representation and then construct the EM mechanism accordingly. This EM mechanism is used to construct conditions and then build an EM-based CDM. During the data synthesis process, we use EM-prioritized condition sampling to guide the EM-based CDM in generating high-quality data. In offline RL, synthetic data can either entirely replace the original dataset or upsample it. In online RL, as the original dataset collected from the envi-

ronment is continuously populated, the hashing-based state representation, EM mechanism, and EM-based CDM need to be periodically updated. The agent then updates its policy using the dataset augmented with the synthetic data. In summary, EMCES is adaptive to both offline and online RL settings.

## 4. Experiments

In this section, we verify the effectiveness of EMCES across different RL settings to demonstrate its ability to synthesize high-quality data. Especially in the offline RL setting, we show that our method can synthesize high-quality data when scaling to vision-based environments. We conduct ablation studies to validate the effectiveness of different components in EMCES. Please note that all results in this paper are summarized over five random seeds.

### 4.1. Main Results

#### 4.1.1. OFFLINE RL

Following prior work (Lu et al., 2023b), we first consider four proprioceptive environments from D4RL (Fu et al., 2020): HalfCheetah, Walker2d, Hopper, and Maze2D. For comparison, we select three classic state-of-the-art offline RL algorithms, including TD3+BC (Fujimoto & Gu, 2021), IQL (Kostrikov et al., 2022) and EDAC (An et al., 2021). We select SynthER as the baseline, and the performance trained by the original datasets from D4RL serves as another baseline. Additionally, we include experimental results of RTDiff (Yang & Wang, 2025), which are provided in the

<sup>3</sup>In the RP-based state representation, each dimension is stored as a 32-bit single-precision floating-point value.

Table 2. Results of all methods on VD4RL environments using BC and DrQ+BC, where each result denotes the mean episode reward and standard deviation.

| Environment        |        | BC                                   |                    |                                      | DrQ+BC              |                                      |                                      |
|--------------------|--------|--------------------------------------|--------------------|--------------------------------------|---------------------|--------------------------------------|--------------------------------------|
|                    |        | Original                             | SynthER            | EMCES                                | Original            | SynthER                              | EMCES                                |
| <i>walker_walk</i> | expert | 879.18 $\pm$ 36.52                   | 901.71 $\pm$ 38.64 | <b>947.43 <math>\pm</math> 10.21</b> | 399.84 $\pm$ 108.88 | <b>717.25 <math>\pm</math> 29.49</b> | 708.67 $\pm$ 24.88                   |
|                    | medrep | <b>121.61 <math>\pm</math> 27.01</b> | 48.79 $\pm$ 13.60  | 93.48 $\pm$ 14.97                    | 169.19 $\pm$ 120.74 | 230.35 $\pm$ 60.54                   | <b>289.77 <math>\pm</math> 32.41</b> |
|                    | medium | 365.11 $\pm$ 51.93                   | 321.81 $\pm$ 37.64 | <b>391.91 <math>\pm</math> 9.67</b>  | 457.05 $\pm$ 15.59  | 447.62 $\pm$ 39.69                   | <b>491.59 <math>\pm</math> 10.52</b> |
|                    | medexp | 448.13 $\pm$ 49.02                   | 389.16 $\pm$ 28.92 | <b>458.92 <math>\pm</math> 53.80</b> | 725.38 $\pm$ 79.87  | 686.36 $\pm$ 81.28                   | <b>790.18 <math>\pm</math> 37.96</b> |
| <i>cheetah_run</i> | expert | 621.23 $\pm$ 72.80                   | 820.34 $\pm$ 29.78 | <b>857.97 <math>\pm</math> 7.96</b>  | 253.47 $\pm$ 32.62  | 266.53 $\pm$ 28.04                   | <b>273.96 <math>\pm</math> 23.99</b> |
|                    | medrep | <b>126.44 <math>\pm</math> 52.49</b> | 27.75 $\pm$ 19.29  | 93.48 $\pm$ 14.97                    | 448.69 $\pm$ 10.17  | 468.59 $\pm$ 9.15                    | <b>475.44 <math>\pm</math> 3.29</b>  |
|                    | medium | 451.63 $\pm$ 27.54                   | 505.40 $\pm$ 11.87 | <b>522.79 <math>\pm</math> 4.00</b>  | 512.12 $\pm$ 25.69  | 561.35 $\pm$ 6.22                    | <b>573.27 <math>\pm</math> 4.29</b>  |
|                    | medexp | 507.24 $\pm$ 19.99                   | 519.78 $\pm$ 5.73  | <b>536.60 <math>\pm</math> 31.58</b> | 350.29 $\pm$ 73.04  | 401.02 $\pm$ 41.28                   | <b>482.72 <math>\pm</math> 44.05</b> |
| <b>average</b>     |        | 440.08                               | 441.84             | <b>487.82</b>                        | 414.50              | 472.38                               | <b>518.00</b>                        |

Appendix due to space constraints. For a fair evaluation of data quality, the agent is trained on synthetic datasets generated by data augmentation methods, each of which matches the size of the original dataset. The resulting performances are reported as normalized scores across all tasks. Detailed hyperparameter settings and the rationale behind their selection are provided in the Appendix, along with the results of hyperparameter sensitivity analysis for  $K$  and  $\beta$ . As shown in Table 1, our method consistently improves the performance of offline RL algorithms across nearly all tasks and often matches or exceeds training on the original datasets, indicating higher-quality synthetic transitions. Furthermore, to illustrate the high-fidelity of synthesized data by EMCES, we measure the faithfulness of generated transitions to environment dynamics in the Appendix, following prior work (Lu et al., 2023b; Wang et al., 2025).

We further scale our method to vision-based environments from the VD4RL (Lu et al., 2023a) benchmark, where the original image observations are of size  $84 \times 84 \times 3$ . We focus on the ‘cheetah-run’ and ‘walker-walk’ environments. Following prior work (Lu et al., 2023b), image observations are encoded into a latent space using a pretrained CNN. All data augmentation methods upsample 5M transitions in the latent space. We choose BC (Lu et al., 2023a) and DrQ+BC (Lu et al., 2023a) as the RL algorithms. As shown in Table 2, EMCES consistently outperforms both the original datasets and SynthER under BC and DrQ+BC.

#### 4.1.2. ONLINE RL

To further demonstrate the robustness and general applicability of EMCES, we also validate its effectiveness in online RL. We select six environments from the DMC suite (Tunyasuvunakool et al., 2020) and the OpenAI Gym Suite (Brockman et al., 2016) to evaluate our method, including quadruped-walk, reacher-hard, cheetah-run, Walker2d, HalfCheetah, and Hopper. We use SAC (Haarnoja et al., 2018) as the RL algorithm to evaluate data augmenta-

tion methods, as it is a classical online RL algorithm. REDQ (Chen et al., 2021), a sample-efficient variant of SAC, serves as an additional baseline. In addition to SynthER (Lu et al., 2023b), we also compare against PGR (Wang et al., 2025), a recent data augmentation method that focuses only on online RL. We denote SAC combined with EMCES, SynthER, and PGR as SAC (EMCES), SAC (SynthER), and SAC (PGR), respectively. All methods are evaluated over 200K online steps for DMC environments and 100K online steps for OpenAI Gym environments. Additional experimental details can be found in the Appendix. The training curves of all methods on six environments are shown in Figure 4. SAC (EMCES) consistently improves sample efficiency and outperforms SAC (SynthER) and SAC (PGR), demonstrating that the synthetic data generated by EMCES is of higher quality.

#### 4.2. Ablation Studies

To investigate the contribution of each component in our method EMCES, we perform a series of ablation studies on the Hopper environment with the ‘medexp’ dataset, employing TD3+BC as the RL algorithm.

**Effect of the conditions** The design of conditions is important for controlling the synthesis process in EMCES, as it dictates how the CDM captures the underlying dynamics of transitions. To show the effect of the conditions, we compare our method with alternative types of conditions, including  $[h(s)]$ ,  $[r + \gamma e(h(s'))]$ , and  $[h(s), d]$ . Table 3 shows that using  $[h(s)]$  and  $[r + \gamma e(h(s'))]$  simultaneously leads to better performance compared to using either one alone. This indicates that combining  $[h(s)]$  and  $[r + \gamma e(h(s'))]$  as the conditions is necessary. Furthermore, replacing  $r + \gamma e(h(s'))$  with the discounted return  $d(s)$  results in decreased performance, although it still surpasses SynthER. This illustrates that although the discounted return  $d(s)$  also contains the cumulative information, the structure of  $r + \gamma e(h(s'))$  aligns more closely with the state-action

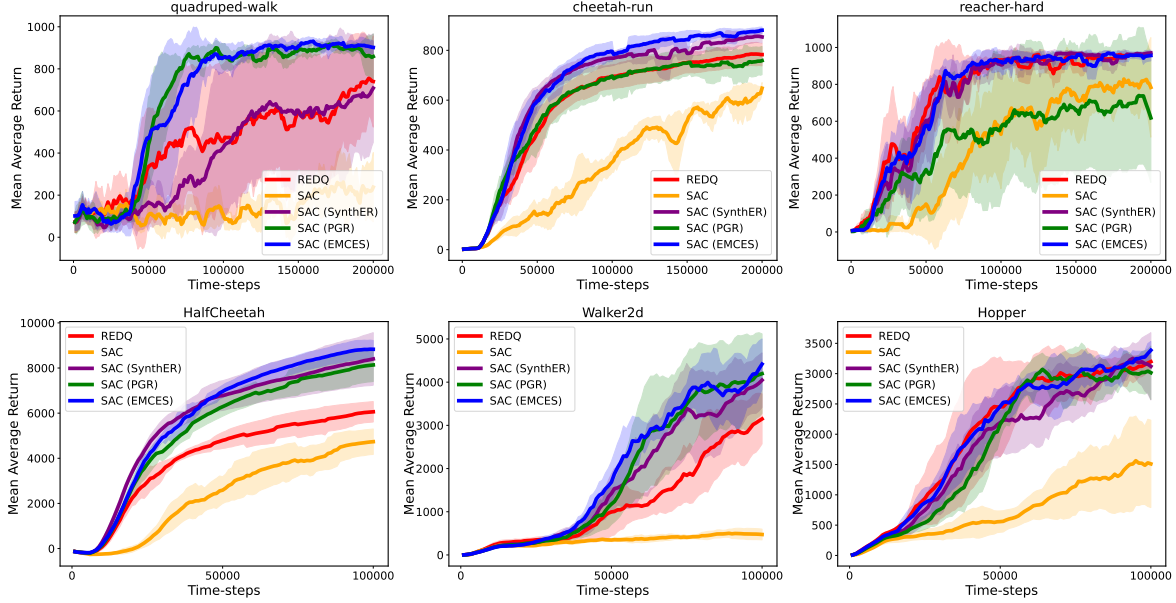


Figure 4. Training curves of SAC (EMCES), SAC (SynthER), SAC (PGR), SAC and REDQ. The solid line represents the mean average return, and the shaded area represents one standard deviation.

value function, making it a more sufficient and effective condition for transition modeling. These results all validate that using  $[h(s), r + \gamma e(h(s'))]$  as the condition can sufficiently reflect the key characteristics of transitions, thereby facilitating effective policy learning.

**Effect of the condition sampling strategies** As for the EM-prioritized condition sampling strategy, we compare it with historical optimal discounted return-based, uniform-based, return-based, and reward-based condition sampling strategies. Specifically, historical optimal discounted return-based, return-based, reward-based condition sampling strategies use  $e(h(s))$ ,  $R$ , and  $r$  to replace  $\delta(x(s, a), h(\cdot))$  in Equation (5), respectively. Uniform condition sampling is equivalent to the EM-prioritized condition sampling strategy when  $\beta = 0$ . As shown in Table 3, the EM-prioritized condition sampling strategy can improve the performance by guiding the synthesis of data with high EMTD-errors, surpassing other sampling strategies. These results verify the necessity of the EM-prioritized condition sampling strategy.

**Effect of the state representations** The choice of state representation decides the efficiency of EM, which further influences the EM-based CDM and the quality of synthesized data. Hence, we summarize the normalized scores of EMCES under different state representations in Table 4. To ensure a fair comparison with hashing-based state representation,  $b_{\text{Grid}}$  is set to 1. As shown in Table 4, EMCES with different state representations all achieve better performance than baselines, especially SynthER. This verifies

Table 3. Performance of EMCES with different conditions and condition sampling strategies on Hopper with the ‘medexp’ dataset. Numbers in parentheses indicate performance improvements relative to SynthER.

| Elements in Condition |                       | Condition Sampling Strategy | Normalized Score          |
|-----------------------|-----------------------|-----------------------------|---------------------------|
| State                 | State-Action Value    |                             |                           |
| EMCES (Our method)    |                       |                             |                           |
| $h(s)$                | $r + \gamma e(h(s'))$ | EMTD-error                  | <b>105.5 ± 3.3 (+9.2)</b> |
| $h(s)$                | -                     | EMTD-error                  | 91.7 ± 11.5 (-4.6)        |
| -                     | $r + \gamma e(h(s'))$ | EMTD-error                  | 91.0 ± 15.9 (-5.3)        |
| $h(s)$                | $d(s)$                | EMTD-error                  | 100.9 ± 8.8 (+4.6)        |
| $h(s)$                | $r + \gamma e(h(s'))$ | $e(h(s))$                   | 103.6 ± 6.1 (+7.4)        |
| $h(s)$                | $r + \gamma e(h(s'))$ | Uniform                     | 91.6 ± 7.5 (-4.7)         |
| $h(s)$                | $r + \gamma e(h(s'))$ | Return                      | 95.2 ± 5.2 (-1.1)         |
| $h(s)$                | $r + \gamma e(h(s'))$ | Reward                      | 99.4 ± 9.8 (+3.1)         |

the effectiveness of the framework of our method EMCES. The hashing-based state representation performs best among all state representations. Here, we conduct experiments on a workstation equipped with a 36-core, 72-thread Intel(R) Xeon(R) Gold 6240 CPU @ 2.60GHz, 377 GB RAM, and 8 NVIDIA GeForce RTX 2080 Ti GPUs. Table 4 also reports the storage and time costs (including retrieval and construction time) required for establishing the EM mechanism. Hashing-based and grid-based state representations can both significantly reduce the storage and time costs. Compared to the RP-based state representation, the hashing-based state representation reduces storage and time costs by approximately 8000-fold and 25.5-fold, respectively. Compared to the grid-based state representation, the hashing-based state representation can help the EM-based CDM gener-

Table 4. Results of EMCES with different state representations (SRs) on Hopper with the ‘medexp’ dataset. The best results are highlighted in bold, while the second-best results are underlined. The number in brackets denotes the performance increase by the variants of EMCES compared to the result achieved by SynthER.

| SR             | Storage Cost (KB) | Time Cost (s) | Normalized Score                         |
|----------------|-------------------|---------------|------------------------------------------|
| RP             | 7806.49           | 7573.88       | $104.0 \pm 8.1$ (+7.7)                   |
| Grid           | <b>0.86</b>       | <b>293.12</b> | $98.2 \pm 12.3$ (+1.9)                   |
| Hashing (Ours) | <u>0.96</u>       | <u>297.12</u> | <b><math>105.5 \pm 3.3</math></b> (+9.2) |

ate higher-quality data to boost the agent’s performance. These findings validate that our proposed hashing-based state representation is essential for guiding the synthesis of high-quality data.

## 5. Conclusion

In this paper, we propose a novel and effective method, EMCES, to enhance the quality of synthetic data. In EMCES, we propose an EM-based CDM, where the EM mechanism is utilized to construct informative and concise conditions. During the data synthesis phase, we propose an EM-prioritized condition sampling strategy to guide the EM-based CDM to synthesize high-quality data. We also propose a hashing-based state representation to improve the efficiency of the EM mechanism and the quality of synthetic data. EMCES is the first work to introduce EM for controlling and further guiding data synthesis. Experimental results show that EMCES can effectively improve the quality of synthetic data.

## Acknowledgements

This work was supported by the NSFC (Grant No. 62506341), the Zhejiang Provincial Natural Science Foundation of China (Grant No. LQN26F020034), the Science Foundation of Zhejiang Sci-Tech University (Grant No. 25232128-Y), the Fundamental Research Funds for the Central Universities and Nanjing University International Collaboration Initiative (Grant No. 020214380129), Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (Grant No. JYB2025XDXM118), and “111 Center” (Grant No. B26023).

## Impact Statement

Real-world RL often suffers from limited data due to high interaction costs and safety constraints, which hinder the deployment of RL agents in practical applications. EMCES addresses this challenge by leveraging episodic memory to guide controllable diffusion models for high-quality data synthesis, significantly improving policy learning efficiency. The broader impact of this research includes reducing the

need for expensive online interactions and making RL more feasible in interaction-sensitive domains such as robotics, autonomous vehicles, and industrial control. From the ethical perspective, our method aims to improve the quality of data synthesis for policy learning and does not inherently introduce bias. Nevertheless, in high-risk scenarios, caution is required to avoid excessive dependence on synthetic data without adequate validation. Overall, this research has the potential to accelerate the adoption of RL in safety-critical and cost-sensitive applications while contributing to responsible and data-efficient machine learning.

## References

- An, G., Moon, S., Kim, J., and Song, H. O. Uncertainty-based offline reinforcement learning with diversified q-ensemble. In *NeurIPS*, 2021.
- Andersen, P. *The hippocampus book*. Oxford university press, Oxford, 2007.
- Bentley, J. L. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- Blundell, C., Uria, B., Pritzel, A., Li, Y., Ruderman, A., Leibo, J. Z., Rae, J. W., Wierstra, D., and Hassabis, D. Model-free episodic control. *CoRR*, abs/1606.04460, 2016.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym. *CoRR*, abs/1606.01540, 2016.
- Chen, H., Lu, C., Ying, C., Su, H., and Zhu, J. Offline reinforcement learning via high-fidelity generative behavior modeling. In *ICLR*, 2023.
- Chen, X., Wang, C., Zhou, Z., and Ross, K. W. Randomized ensembled double q-learning: Learning fast without a model. In *ICLR*, 2021.
- Dong, Z., Hao, J., Yuan, Y., Ni, F., Wang, Y., Li, P., and Zheng, Y. Diffuserlite: Towards real-time diffusion planning. In *NeurIPS*, 2024.
- Fan, K., Zhang, J., Zhang, X., Wu, Y., Cao, J., Zhou, Y., and Ma, J. Safety-polarized and prioritized reinforcement learning. In *ICML*, 2025.
- Fu, J., Kumar, A., Nachum, O., Tucker, G., and Levine, S. D4RL: datasets for deep data-driven reinforcement learning. *CoRR*, abs/2004.07219, 2020.
- Fujimoto, S. and Gu, S. S. A minimalist approach to offline reinforcement learning. In *NeurIPS*, 2021.

- Fujimoto, S., van Hoof, H., and Meger, D. Addressing function approximation error in actor-critic methods. In *ICML*, 2018.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial nets. In *NeurIPS*, 2014.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML*, 2018.
- Hafner, D., Lillicrap, T. P., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. In *ICLR*, 2020.
- He, H., Bai, C., Xu, K., Yang, Z., Zhang, W., Wang, D., Zhao, B., and Li, X. Diffusion model is an effective planner and data synthesizer for multi-task reinforcement learning. In *NeurIPS*, 2023a.
- He, R., Sun, S., Yu, X., Xue, C., Zhang, W., Torr, P. H. S., Bai, S., and Qi, X. Is synthetic data from generative models ready for image recognition? In *ICLR*, 2023b.
- Ho, J. and Salimans, T. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- Horgan, D., Quan, J., Budden, D., Barth-Maron, G., Hessel, M., van Hasselt, H., and Silver, D. Distributed prioritized experience replay. In *ICLR*, 2018.
- Hu, H., Ye, J., Zhu, G., Ren, Z., and Zhang, C. Generalizable episodic memory for deep reinforcement learning. In *ICML*, 2021.
- Hu, J., Jiang, Y., and Weng, P. Revisiting data augmentation in deep reinforcement learning. In *ICLR*, 2024.
- Huang, V., Ley, T., Vlachou-Konchylaki, M., and Hu, W. Enhanced experience replay generation for efficient reinforcement learning. *CoRR*, abs/1705.08245, 2017.
- Huang, X., Liu, X., Zhang, E., Yu, T., and Li, S. Offline-to-online reinforcement learning with classifier-free diffusion generation. In *ICML*, 2025.
- Janner, M., Du, Y., Tenenbaum, J. B., and Levine, S. Planning with diffusion for flexible behavior synthesis. In *ICML*, 2022.
- Karras, T., Aittala, M., Aila, T., and Laine, S. Elucidating the design space of diffusion-based generative models. In *NeurIPS*, 2022.
- Kingma, D. P. and Welling, M. Auto-encoding variational bayes. In *ICLR*, 2014.
- Kong, W. and Li, W. Isotropic hashing. In *NeurIPS*, 2012.
- Kononenko, I. and Kukar, M. *Machine Learning and Data Mining*. Horwood Publishing, 2007.
- Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit q-learning. In *ICLR*, 2022.
- Kuznetsov, I. and Filchenkov, A. Solving continuous control with episodic memory. In *IJCAI*, 2021.
- Laskin, M., Lee, K., Stooke, A., Pinto, L., Abbeel, P., and Srinivas, A. Reinforcement learning with augmented data. In *NeurIPS*, 2020.
- Lee, J., Yun, S., Yun, T., and Park, J. GTA: generative trajectory augmentation with guidance for offline reinforcement learning. In *NeurIPS*, 2024.
- Levine, S., Kumar, A., Tucker, G., and Fu, J. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *CoRR*, abs/2005.01643, 2020.
- Li, G., Shan, Y., Zhu, Z., Long, T., and Zhang, W. Diffstitch: Boosting offline reinforcement learning with diffusion-based trajectory stitching. In *ICML*, 2024.
- Li, Z., Zhu, D., Hu, Y., Xie, X., Ma, L., Zheng, Y., Song, Y., Chen, Y., and Zhao, J. Neural episodic control with state abstraction. In *ICLR*, 2023.
- Lian, S., Ma, Y., Liu, J., Zheng, Y., and Meng, Z. HIPODE: enhancing offline reinforcement learning with high-quality synthetic data from a policy-decoupled approach. *CoRR*, abs/2306.06329, 2023.
- Lin, Z., Zhao, T., Yang, G., and Zhang, L. Episodic memory deep q-networks. In *IJCAI*, 2018.
- Liu, J., Ma, Y., Hao, J., Hu, Y., Zheng, Y., Lv, T., and Fan, C. A trajectory perspective on the role of data sampling techniques in offline reinforcement learning. In *AAMAS*, 2024.
- Lu, C., Ball, P. J., Rudner, T. G. J., Parker-Holder, J., Osborne, M. A., and Teh, Y. W. Challenges and opportunities in offline reinforcement learning from visual observations. *Transaction of Machine Learning Research*, 2023a.
- Lu, C., Ball, P. J., Teh, Y. W., and Parker-Holder, J. Synthetic experience replay. In *NeurIPS*, 2023b.
- Lu, C., Chen, H., Chen, J., Su, H., Li, C., and Zhu, J. Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning. In *ICML*, 2023c.

- Ma, X. and Li, W.-J. State-based episodic memory for multi-agent reinforcement learning. *Machine Learning*, 112(12):5163–5190, 2023.
- Ma, X. and Li, W.-J. Weighting online decision transformer with episodic memory for offline-to-online reinforcement learning. In *ICRA*, 2024.
- Ma, X., Yang, Y., Hu, H., Yang, J., Zhang, C., Zhao, Q., Liang, B., and Liu, Q. Offline reinforcement learning with value-based episodic memory. In *ICLR*, 2022.
- Miki, T., Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., and Hutter, M. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62):eabk2822, 2022.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M. A., Fidjeland, A., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.
- Pritzel, A., Uria, B., Srinivasan, S., Badia, A. P., Vinyals, O., Hassabis, D., Wierstra, D., and Blundell, C. Neural episodic control. In *ICML*, 2017.
- Ren, Z., Zhu, G., Hu, H., Han, B., Chen, J., and Zhang, C. On the estimation bias in double q-learning. In *NeurIPS*, 2021.
- Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. In *ICLR*, 2016.
- Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T. P., and Silver, D. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T. P., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- Sinha, S., Mandlekar, A., and Garg, A. S4RL: surprisingly simple self-supervision for offline reinforcement learning in robotics. In *CoRL*, 2021.
- Squire, L. R. Memory systems of the brain: a brief history and current perspective. *Neurobiology of Learning and Memory*, 82(3):171–177, 2004.
- Sutton, R. S. and Barto, A. G. *Reinforcement learning: An introduction*. MIT press, 2018.
- Tang, H., Houthoofd, R., Foote, D., Stooke, A., Chen, X., Duan, Y., Schulman, J., Turck, F. D., and Abbeel, P. #exploration: A study of count-based exploration for deep reinforcement learning. In *NeurIPS*, 2017.
- Trabucco, B., Doherty, K., Gurinas, M., and Salakhutdinov, R. Effective data augmentation with diffusion models. In *ICML*, 2024.
- Tunyasuvunakool, S., Muldal, A., Doron, Y., Liu, S., Bohez, S., Merel, J., Erez, T., Lillicrap, T. P., Heess, N., and Tassa, Y. dm\_control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020.
- van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. In *AAAI*, 2016.
- Wang, J., Li, W., Jiang, H., Zhu, G., Li, S., and Zhang, C. Offline reinforcement learning with reverse model-based imagination. In *NeurIPS*, 2021.
- Wang, R., Frans, K., Abbeel, P., Levine, S., and Efros, A. A. Prioritized generative replay. In *ICLR*, 2025.
- Wang, Z., Bapst, V., Heess, N., Mnih, V., Munos, R., Kavukcuoglu, K., and de Freitas, N. Sample efficient actor-critic with experience replay. In *ICLR*, 2017.
- Wang, Z., Hunt, J. J., and Zhou, M. Diffusion policies as an expressive policy class for offline reinforcement learning. In *ICLR*, 2023.
- Yang, Q. and Wang, Y. Atradiff: Accelerating online reinforcement learning with imaginary trajectories. In *ICML*, 2024.
- Yang, Q. and Wang, Y. Rtdiff: Reverse trajectory synthesis via diffusion for offline reinforcement learning. In *ICLR*, 2025.
- Yang, Z., Moerland, T. M., Preuss, M., and Plaat, A. Continuous episodic control. In *CoG*, 2023.
- Zhang, L., Rao, A., and Agrawala, M. Adding conditional control to text-to-image diffusion models. In *ICCV*, 2023.
- Zhang, Q., Fang, X., Xu, K., Zhao, W., Li, H., and Zhao, D. High-quality synthetic data is efficient for model-based offline reinforcement learning. In *IJCNN*, 2024.

Zheng, L., Chen, J., Wang, J., He, J., Hu, Y., Chen, Y., Fan, C., Gao, Y., and Zhang, C. Episodic multi-agent reinforcement learning with curiosity-driven exploration. In *NeurIPS*, 2021.

Zhu, Z., Zhao, H., He, H., Zhong, Y., Zhang, S., Yu, Y., and Zhang, W. Diffusion models for reinforcement learning: A survey. *CoRR*, abs/2311.01223, 2023.

Zhu, Z., Liu, M., Mao, L., Kang, B., Xu, M., Yu, Y., Ermon, S., and Zhang, W. Madiff: Offline multi-agent learning with diffusion models. In *NeurIPS*, 2024.

## A. Background

### A.1. Episodic Memory

Humans can rapidly exploit newly discovered high-reward opportunities due to the hippocampus, which stores episodic memory (EM) (Squire, 2004; Andersen, 2007). Inspired by this, recent works integrate the EM mechanism into RL to help agents utilize valuable past experiences stored in EM through a non-parametric approach, thereby improving the sample efficiency (Blundell et al., 2016; Pritzel et al., 2017; Lin et al., 2018; Kuznetsov & Filchenkov, 2021; Zheng et al., 2021; Ma & Li, 2023). EM mechanisms can be categorized into state-action-based (Blundell et al., 2016; Pritzel et al., 2017; Lin et al., 2018) and state-based EM mechanisms (Ma & Li, 2023; Yang et al., 2023). In particular, the state-based EM mechanism has not only proven its effectiveness but also demonstrated its high computational efficiency. Hence, we adopt the state-based EM mechanism in this paper. Specifically, the EM mechanism maintains a lookup table  $Q^S$ , where each entry  $Q^S(h(s))$  stores a tuple  $(h(s), e(h(s)))$ , with  $h(s)$  serving as the index for lookup. Please note that  $h(s)$  represents the encoded state of  $s$ , where  $h(\cdot) : \mathcal{S} \rightarrow \mathbb{R}^K$  is a state representation function, and  $K$  denotes the dimension of the encoded state.  $e(h(s))$  stores the historical optimal discounted return associated with  $h(s)$ . The EM mechanism has two operators: lookup and update. The lookup operator retrieves  $e(h(s))$ , the historical optimal discounted return for a given state  $s$ , by using  $h(s)$  as an index in the lookup table  $Q^S$ . Given any trajectory  $\tau = \{(s_t, a_t, r_t, s'_t)\}_{t=0}^T$ , we compute the discounted return  $d(s_t) = \sum_{i=t}^T \gamma^{i-t} r_i$  at time-step  $t \in \{0, \dots, T\}$ , where  $s_t, a_t, r_t$ , and  $s'_t$  are the corresponding state, action, reward, and next state at time-step  $t \in \{0, \dots, T\}$ , respectively.  $d(s_t)$  can be abbreviated as  $d_t$ . And then the lookup table  $Q^S$  is updated with the set  $\{(h(s_t), d_t)\}_{t=0}^T$  based on the following rules:

$$Q^S(h(s_t)) \leftarrow \begin{cases} (h(s_t), d_t), & \text{if } Q^S(h(s_t)) \notin Q^S; \\ (h(s_t), e(h(s_t))), & \text{if } Q^S(h(s_t)) \in Q^S \text{ and } e(h(s_t)) \geq d_t; \\ (h(s_t), d_t), & \text{if } Q^S(h(s_t)) \in Q^S \text{ and } e(h(s_t)) < d_t. \end{cases}$$

The lookup operator can implicitly merge multiple trajectories, allowing the lookup table to store the historical optimal discounted return for each  $h(s)$ . Furthermore, researchers have proposed a class of parametric EM mechanisms (Hu et al., 2021; Ma et al., 2022). However, these parametric EM mechanisms are inherently limited by their reliance on Q-value functions. To the best of our knowledge, SfBC (Chen et al., 2023) is the only work that integrates both the DM and the EM mechanism, but it does not utilize the EM mechanism to control DMs. Hence, our work is the first to introduce the EM mechanism as a control mechanism for DMs and guide the data synthesis in RL.

#### A.1.1. STATE REPRESENTATION IN EM

Although state representation has been studied in RL (Schrittwieser et al., 2020; Hafner et al., 2020; Tang et al., 2017), state representation for EM mechanisms is still naive. The default state representation is random projection (RP), a classical dimensionality reduction technique (Kononenko & Kukar, 2007). Each state is typically stored as  $K$  single-precision (32-bit) floating-point values, and the total number of encoded states  $n_{RP}$  is approximately  $N$ , where  $N$  denotes the total number of transitions in the dataset. Hence, the storage complexity, retrieval time complexity, and construction time complexity of corresponding EM mechanisms<sup>4</sup> are  $\mathcal{O}(32Kn_{RP})$ ,  $\mathcal{O}(\log n_{RP})$  and  $\mathcal{O}(n_{RP} \log n_{RP})$ , respectively. However, real-valued encoded states are unlikely to repeat (Li et al., 2023), which leads to inefficient EM updates and high storage, retrieval, and construction complexities. However, real-valued encoded states are unlikely to repeat (Li et al., 2023), which leads to inefficient EM updates and high storage, retrieval, and construction complexities. Grid-based state representation for EM mechanisms (Li et al., 2023) is proposed to address these issues, where each dimension of states is encoded by discretizing the state space into a grid-like structure. Grid-based state representation groups states that share the same characteristics into a single cluster, enabling more effective EM updates while reducing the complexities of storage, retrieval, and construction. Each state would be stored as  $b_{Grid}$ -bit integers ( $b_{Grid} < 32$ , typically) and the number of encoded states  $n_{Grid}$  would be smaller than  $N$ . The storage, retrieval time, and construction time complexities of corresponding EM mechanisms are  $\mathcal{O}(b_{Grid}Kn_{Grid})$ ,  $\mathcal{O}(\log n_{Grid})$  and  $\mathcal{O}(n_{Grid} \log n_{Grid})$ , respectively. Since  $n_{Grid} \ll N$ , the storage, retrieval time, and construction time complexities of EM mechanisms with a grid-based state representation are lower than those with the RP-based state representation. However, the above state representations are both data-independent and ignore the underlying

<sup>4</sup>In this paper, we only focus on the storage complexity of encoded states in EM mechanisms and the lookup tables in EM mechanisms are implemented by the KD-tree data structure (Bentley, 1975).

---

**Algorithm 1** Training Process of EMCES
 

---

**Input:** an initialized diffusion model  $M_\phi$ , a dataset  $D$ , an empty lookup table  $Q^S$ ;  
**Output:**  $M_\phi, C$ ;  
 Use IsoHash (Kong & Li, 2012) to learn  $h(\cdot)$  based on  $D$ ;  
**for** each transition  $(s, a, r, s')$  **do**  
     Compute discounted return  $d$  and hash code  $h(s)$  for  $s \in D$ ;  
     Use  $(h(s), d)$  to update  $Q^S$  based on (1);  
**end for**  
 Compute the condition set  $C$  based on  $Q^S, h(\cdot)$  and (3);  
 Compute  $\delta(x(s, a), h(\cdot))$  based on (4),  $\forall x(s, a) \in D$ ;  
 Train the EM-based CDM  $M_\phi$  by using  $D$  and  $C$ ;

---



---

**Algorithm 2** Data Synthesis Process of EMCES
 

---

**Input:** a condition set  $C$ , an EM-based CDM  $M_\phi$ , number of synthetic traintitions  $N_{\text{EMCES}}$ ;  
**Output:**  $D_{\text{EMCES}}$ ;  
 Compute the probability based on (5) for each condition  $y(x(s, a), h(\cdot)) \in C$ ;  
 Sample  $N_{\text{EMCES}}$  conditions from  $C$  based on the probability;  
 Use the sampled conditions and  $M_\phi$  to generate transitions  $D_{\text{EMCES}} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^{N_{\text{EMCES}}}$ ;

---

data distribution, leading to inefficient updates in EM mechanisms. These limitations motivate us to design an effective state representation for EM mechanisms.

## A.2. Controllable Diffusion Model

To enable control over the synthesis process, we incorporate the controllable diffusion model (CDM) (Ho & Salimans, 2021) into our work. Let  $\mathcal{D}(x)$  (also denoted as  $\mathcal{D}_0(x)$ ) represents the data distribution and  $\mathcal{D}_k(x_k)$  denote a series of time-dependent distributions obtained by injecting *i.i.d.* Gaussian noise to data from  $\mathcal{D}_0(x)$  during the forward noising process. Specifically, at time step  $k$ ,  $\mathcal{D}_k(x_k|x) = \mathcal{N}(x, \sigma_k^2 I)$ , where  $\sigma_k$  is the noise level. Given a sample  $x \sim \mathcal{D}(x)$ , let  $y(x)$  be its corresponding condition. In the backward denoising process, Karras et al. (2022) propose to train a denoiser  $M_\phi(x, y; \sigma)$  with parameters  $\phi$  on an L2 denoising objective, formulated as:

$$\min_{\phi} \mathbb{E}_{x \sim \mathcal{D}, \sigma, \epsilon \sim \mathcal{N}(0, \sigma^2 I)} \|M_\phi(x + \epsilon, y(x); \sigma) - x\|_2^2. \quad (6)$$

The design of the condition  $y(x)$  is critical (Zhang et al., 2023), as a well-designed  $y(x)$  can ensure the generated data aligns with the desired features or structure. Conversely, a poorly designed  $y(x)$  might cause the model to generate irrelevant or poor-fidelity outputs. Furthermore, for details on the training and sampling processes, we refer readers to the original paper (Karras et al., 2022).

## B. Algorithms of EMCES

EMCES typically has two processes: the training and data synthesis processes. During the training process, we obtain a hashing-based state representation and then construct the EM mechanism based on the existing dataset. The training process is summarized in Algorithm 1. During the data synthesis process, we prioritize sampling conditions based on EMTD-errors to guide the EM-based CDM in synthesizing high-quality data  $D_{\text{EMCES}}$ . The data synthesis process is summarized in Algorithm 2.

## C. Hyperparameters

We list all the related hyperparameters here. In addition, we will release our code upon acceptance. The global parameters encompass the architecture and optimization settings for the denoising network, as well as the configuration of the diffusion process. The values of these global parameters are shown in Table 5 and applied in all environments and methods unless otherwise specified.

Table 5. Hyperparameter settings for the denoising network, optimizer, and diffusion process in all methods.

| Component                   | Hyperparameter                 | Value                                                  |
|-----------------------------|--------------------------------|--------------------------------------------------------|
| MLP<br>Denoising<br>Network | number of layers               | 6                                                      |
|                             | width                          | 1024                                                   |
|                             | activation function            | ReLU                                                   |
| Optimizer                   | optimizer                      | Adam                                                   |
|                             | learning rate                  | $3 \times 10^{-4}$                                     |
|                             | learning rate schedule         | Cosine annealing                                       |
|                             | training steps                 | 100K                                                   |
|                             | batch size                     | 256 for online and<br>medium-replay,<br>1024 otherwise |
| Diffusion<br>Process        | diffusion steps                | 128                                                    |
|                             | $\sigma_{\min}, \sigma_{\max}$ | [0.002, 80]                                            |
|                             | $S_{\min}, S_{\max}$           | [0.05, 50]                                             |
|                             | $S_{\text{noise}}$             | 1.003                                                  |
|                             | $S_{\text{churn}}$             | 80                                                     |

 Table 6. Hyperparameter settings of  $K$  and  $\beta$  for different environments from D4RL. Please note that  $K$  represents the dimension of the state space and  $K \leq F$ .

| Environment ( $F$ ) | Task   | Hyperparameters |         |
|---------------------|--------|-----------------|---------|
|                     |        | $K$             | $\beta$ |
| HalfCheetah (17)    | random | 8               | 1e-4    |
|                     | medrep | 8               | 1e-2    |
|                     | medium | 8               | 1e-2    |
|                     | medexp | 16              | 1e-4    |
| Walker2d (17)       | random | 8               | 1e-4    |
|                     | medrep | 8               | 1e-2    |
|                     | medium | 8               | 1e-2    |
|                     | medexp | 8               | 1e-2    |
| Hopper (11)         | random | 11              | 1e-4    |
|                     | medrep | 11              | 1e-4    |
|                     | medium | 11              | 1e-4    |
|                     | medexp | 11              | 1e-4    |
| Maze2d (4)          | umaze  | 4               | 1e-2    |
|                     | medium | 4               | 1e-4    |
|                     | large  | 4               | 1e-4    |

Furthermore, in our method, two key environment-specific hyperparameters are introduced: the dimension of the encoded state,  $K$ , and the weighting coefficient,  $\beta$ , for the EM-prioritized condition sampling strategy.  $K$  controls the size of the compressed representation stored in EM.  $\beta \in [0, 1]$  determines the strength of prioritization based on EMTD-errors in the EM-prioritized condition sampling strategy;  $\beta = 0$  corresponds to uniform sampling, while larger values give higher importance to transitions with larger EMTD-error. Since  $K \leq F$  and  $F$  varies across environments (as shown in Table 6), we empirically select  $K$  from  $\{8, 16\}$  for HalfCheetah and Walker2d and select  $K$  from  $\{8, 11\}$  for Hopper. As for Maze2d, its dimension of state space  $F$  is 4, so we choose  $K = 4$ . We also empirically select  $\beta$  from  $\{1e-2, 1e-4\}$  for each task. The values used in all D4RL experiments are summarized in Table 6. And we maintain consistency in hyperparameters across all tasks unless otherwise specified. As for the VD4RL environments, the dimensions of the state space  $F$  are 24 and 12 for DrQ+BC and BC, respectively. Hence,  $K$  is set to 16 and 8 for DrQ+BC and BC, respectively. And  $\beta$  is set to 1e-4. As for the online RL setting, we set  $K = 6$  and  $\beta = 1e-2$ .

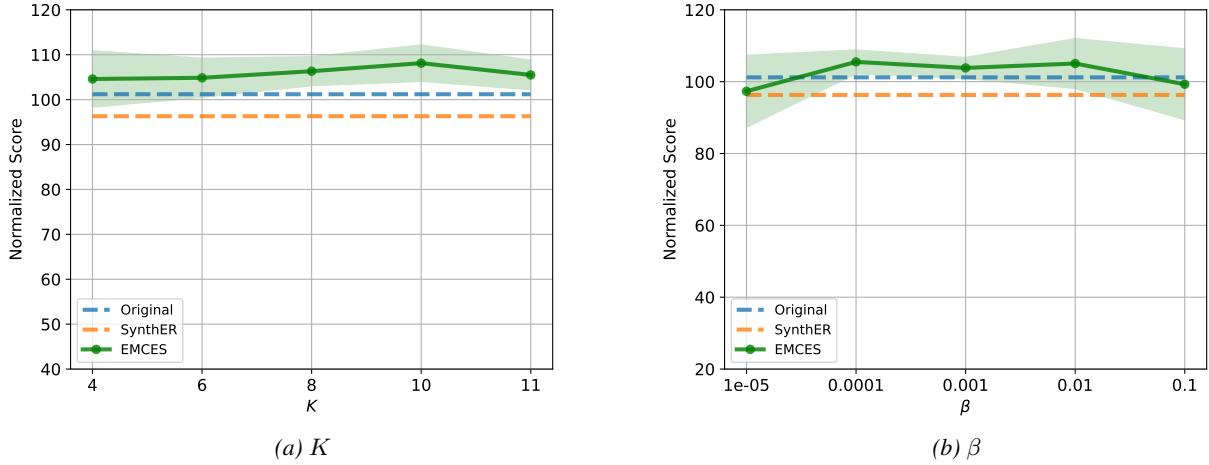


Figure 5. Sensitivity analysis of EMCES on Hopper with the ‘medexp’ dataset. Left: performance with different  $K$ . Right: performance with different  $\beta$ .

### C.1. Sensitivity to Hyper-parameters

To investigate the effect of hyperparameters, we choose the Hopper with the ‘medexp’ dataset as the evaluation task and TD3+BC as the RL algorithm. In EMCES, the hyperparameters include the length of hash codes  $K$  and the priority level of the EMTD-error  $\beta$ .  $K$  affects the granularity of state representations in the embedding space, and a larger value of  $K$  enforces finer distinctions between similar states. For Hopper with the ‘medexp’ dataset, where the state space has a dimension of 11,  $K$  is chosen from  $\{4, 6, 8, 10, 11\}$  when  $\beta = 1e-4$ . Figure 5a presents the results of different values. We can observe that EMCES performs well when  $K$  is chosen from  $\{8, 10, 11\}$ . The consistent performance of EMCES across  $K \in \{8, 10, 11\}$  suggests that our method is relatively insensitive to the hyperparameter  $K$ .

$\beta$  decides the priority level of the EMTD-errors in the data synthesis process. A larger  $\beta$  would tend to guide the synthesis of transitions with higher EMTD-errors. As  $\beta$  approaches 0, the sampling process gradually converges to uniform sampling. Performance comparison across different values of  $\beta$  with fixed  $K = 11$  on Hopper is shown in Figure 5b, where  $\beta \in \{1e-5, 1e-4, 1e-3, 1e-2, 1e-1\}$ . We can find that our method performs better than baselines when  $\beta \in \{1e-4, 1e-3, 1e-2\}$ . These results verify that our method can improve the quality of synthetic data and be relatively insensitive to the hyperparameter  $\beta$ .

## D. Results in Offline and Online RL

### D.1. Offline RL

As the main paper claimed, we also compare our method, EMCES, with RTDiff (Yang & Wang, 2025) in offline RL. For the trajectory-based data augmentation method, RTDiff, we follow the original paper and set the width of the MLP denoising network to 4096 and the training steps to  $10^6$ . We performed a coarse search of hyperparameters for length from  $\{2, 3, 5, 10\}$  and found that when length is set to 3, it is a reasonable choice in most environments. For Hopper and Halfcheetah environments with the ‘medexp’ dataset, we found that performance is best when the length is set to 2. The results of all data augmentation methods on D4RL environments are shown in Table 7. From Table 7, we can observe that our method outperforms all baselines in most environments. This verifies that our method can effectively improve the quality of the synthetic data.

### D.2. Online RL

#### D.2.1. TRAINING DETAILS IN ONLINE RL

In online RL, we use SAC (Haarnoja et al., 2018) as the RL algorithm to evaluate data augmentation methods, as it is a classical online RL algorithm. SAC sets the update-to-data (UTD) ratio to 1 (1 update for each transition collected). We also

Table 7. Overall results on D4RL environments using selected state-of-the-art offline RL algorithms. Note that for each task, all methods are evaluated on datasets of equal size.

| Environ-<br>ment | Task   | TD3+BC     |            |              | IQL       |           |              | EDAC        |            |              |
|------------------|--------|------------|------------|--------------|-----------|-----------|--------------|-------------|------------|--------------|
|                  |        | RTDiff     | SynthER    | EMCES        | RTDiff    | SynthER   | EMCES        | RTDiff      | SynthER    | EMCES        |
| Half<br>Cheetah  | random | 9.9 ±0.5   | 11.5±0.6   | 12.5±0.4     | 6.4±4.2   | 13.8±1.2  | 13.7±1.8     | 11.9±5.0    | 16.5±5.9   | 18.6±1.2     |
|                  | medrep | 41.8±0.5   | 44.5±0.7   | 45.2±0.6     | 43.6±0.4  | 44.3±0.3  | 44.3±0.2     | 48.5±0.7    | 55.8±2.4   | 56.5±0.5     |
|                  | medium | 46.3±0.1   | 48.7±0.3   | 48.8 ± 0.2   | 50.5±0.5  | 48.9±0.3  | 49.3±0.1     | 56.4±0.9    | 63.9±2.5   | 68.2±1.0     |
|                  | medexp | 93.4±1.3   | 76.2±13.4  | 90.7±3.3     | 95.6±0.4  | 92.1±5.2  | 93.8±1.2     | 83.4±3.4    | 63.7±7.6   | 75.8±10.0    |
| Walker2d         | random | 1.6 ±1.4   | 1.8±2.4    | 4.0±2.5      | 5.3±0.7   | 3.4±1.1   | 3.7±0.3      | 5.1±7.8     | 10.8±8.8   | 21.6±1.1     |
|                  | medrep | 67.7±12.2  | 85.9±1.8   | 85.7±2.3     | 77.2±5.8  | 82.0±8.8  | 82.4±2.1     | 80.2±0.6    | 74.9±1.4   | 72.6±0.3     |
|                  | medium | 82.2±1.8   | 84.1±0.3   | 84.2±1.1     | 64.2±25.7 | 83.0±4.0  | 86.1±2.1     | 28.0±16.8   | 88.2±1.0   | 89.4±1.0     |
|                  | medexp | 110.4±0.3  | 110.5±0.5  | 110.7±0.3    | 112.3±0.4 | 111.2±1.0 | 111.7±0.5    | 109.6±0.6   | 111.6±1.0  | 112.9±0.6    |
| Hopper           | random | 8.4±0.1    | 7.8±0.6    | 14.1±8.7     | 7.2±0.2   | 7.9±1.0   | 7.4±0.2      | 6.2±0.8     | 5.2±1.1    | 16.4±12.2    |
|                  | medrep | 71.7±14.4  | 58.9±14.0  | 68.5±14.1    | 19.3±13.6 | 42.3±37.9 | 101.0±0.9    | 100.6±0.6   | 101.0±0.7  | 99.6±0.8     |
|                  | medium | 60.1±5.8   | 57.3±2.6   | 61.5±2.8     | 63.0±2.6  | 65.9±2.3  | 67.4±2.7     | 102.3±0.2   | 100.0±4.3  | 100.4±1.4    |
|                  | medexp | 102.7±6.6  | 96.3±13.5  | 105.5±3.3    | 104.7±1.7 | 85.2±27.0 | 102.9±8.8    | 89.3±15.5   | 99.8±14.8  | 105.0±7.0    |
| <b>sum</b>       |        | 696.2      | 683.5      | <b>731.4</b> | 649.3     | 680.0     | <b>763.7</b> | 721.5       | 791.4      | <b>837.0</b> |
| maze2d           | umaze  | 37.6±9.2   | 37.0±26.8  | 41.9±3.1     | 40.3±1.7  | 40.4±2.4  | 40.4±1.2     | 89.2± 7.1   | 92.1±11.0  | 92.2±3.6     |
|                  | medium | 56.0±4.6   | 50.8±32.8  | 87.7±47.3    | 34.4±0.9  | 35.0±1.7  | 35.1±0.7     | 60.4± 11.3  | 61.7±16.1  | 95.6±27.9    |
|                  | large  | 119.9±52.6 | 105.5±52.3 | 102.3±44.7   | 64.1±0.9  | 59.4±6.3  | 66.1±2.2     | 132.0± 24.7 | 123.2±25.2 | 144.7±20.2   |
| <b>sum</b>       |        | 213.5      | 193.3      | <b>231.9</b> | 138.8     | 134.8     | <b>141.6</b> | 281.6       | 277.0      | <b>332.5</b> |

include REDQ (Chen et al., 2021), a sample-efficient variant of SAC, as a baseline with a UTD ratio of 20. SynthER (Lu et al., 2023b), PGR (Wang et al., 2025) and RTDiff (Yang & Wang, 2025) are included as baselines. These data augmentation methods retrain their diffusion model for every 10K transitions collected from the environment, generate 1M synthetic transitions, and then sample an equal mixture of original and synthetic data for training. In SAC with all data augmentation methods, the UTD ratio is set to 20. We implement PGR (Wang et al., 2025) using the code from its author<sup>5</sup>, where the denoising network is larger than that of our method and other baselines. As for RTDiff (Yang & Wang, 2025), we also set the length to 3. For a fair comparison, the other settings of RTDiff are identical to those of our method and SynthER. We evaluate all methods over 200K online steps for DMC environments and 100K online steps for OpenAI Gym environments.

## D.2.2. RESULTS IN ONLINE RL

Figure 6 summarizes the training curves of all methods. We observe that our method, EMCES, still outperforms all baselines. This demonstrates again that the synthetic data generated by our method is of higher quality than that generated by other data augmentation methods.

Furthermore, we also compare the model size, memory usage, and training time of all methods in Table 8. Here, the experiments were conducted on the Hopper environment for 30K steps, and the diffusion model was retrained every 10K steps. Note that for PGR\*, we replaced the original denoising network with a standardized lightweight architecture to align with other baselines. We conduct experiments on a workstation equipped with a 36-core, 72-thread Intel(R) Xeon(R) Gold 6240 CPU @ 2.60GHz, 377 GB RAM, and 8 NVIDIA GeForce RTX 2080 Ti GPUs. We can find that the memory usage and training time of our method do not incur extra costs.

## E. More Results

The controllable diffusion model is well-known to improve data-fidelity in the generative modeling literature (Ho & Salimans, 2021). We follow the prior work (Lu et al., 2023b) and compare the faithfulness of generated transitions to the real environment dynamics. Given a synthesized transition  $(s, a, r, s')$ , we roll out the action  $a$  given the current state  $s$  in the environment simulator to obtain the ground truth next state  $s'$  and reward. We then measure the mean-squared error (MSE) between the ground truth and synthesized values. In offline RL, we compare our method with SynthER on the Hopper with ‘medexp’ dataset and roll out 10K transitions for each method. As shown in Figure 7, the fidelity of data generated by SynthER and EMCES is similar. Hence, the advantage of our method does not lie in generating high-fidelity transitions

<sup>5</sup><https://pgenreplay.github.io>

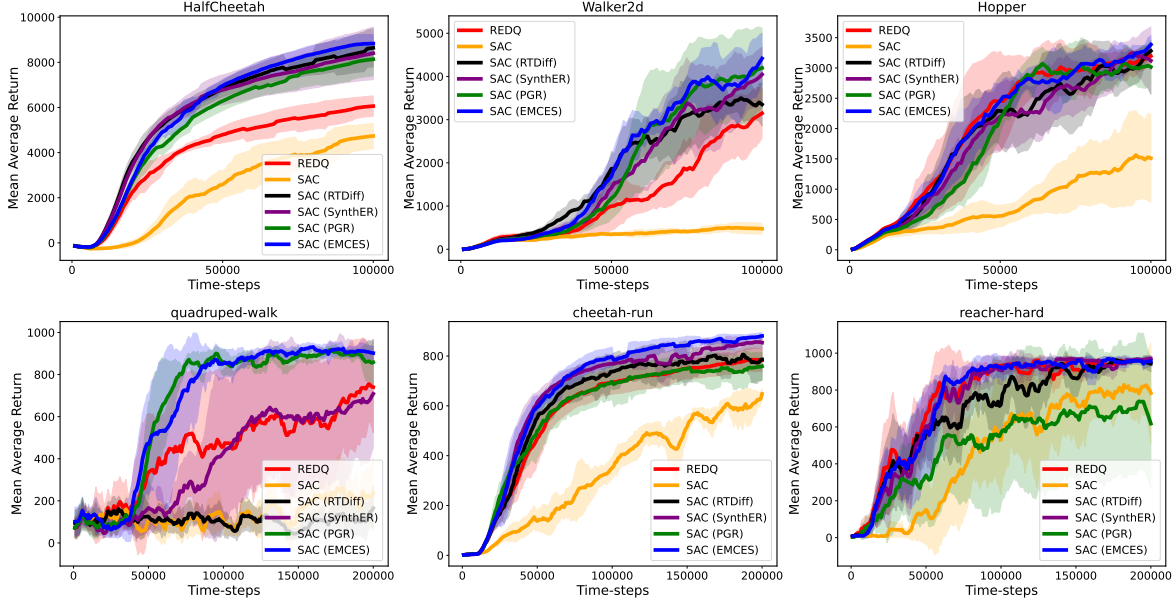


Figure 6. Training curves of all methods. The solid line represents the mean average return, and the shaded area represents one standard deviation.

Table 8. Comparison of model size, memory usage, and training efficiency among all methods in online RL.

|                                     | REDQ | SAC (SynthER) | SAC (PGR) | SAC (PGR*) | SAC (RTDiff) | SAC (EMCES) |
|-------------------------------------|------|---------------|-----------|------------|--------------|-------------|
| Model Size ( $\times 10^6$ params.) | 1.75 | 7.01          | 27.68     | 8.21       | 7.18         | 7.08        |
| Generation VRAM (GB)                | —    | 3.47          | 6.24      | 3.68       | 3.70         | 3.60        |
| Train Time, hours (Diffusion)       | —    | 0.83          | 1.04      | 0.90       | 0.88         | 0.84        |
| Generation Time, hours              | —    | 0.29          | 1.57      | 0.45       | 0.31         | 0.30        |
| Train Time, hours (RL)              | 2.99 | 0.79          | 0.94      | 0.97       | 0.80         | 0.78        |
| Train Time, hours (Total)           | 2.99 | 1.91          | 3.55      | 2.32       | 1.99         | 1.92        |

from the perspective of generative modeling; instead, EMCES focuses on generating high-value transitions to effectively facilitate policy learning.

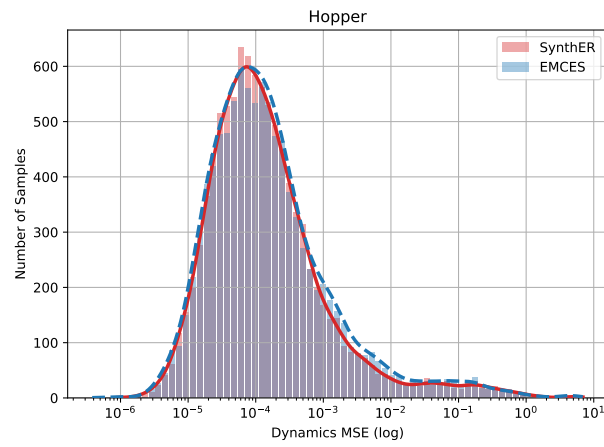


Figure 7. Histograms of dynamics MSE values on Hopper for SynthER and EMCES.